

ECS 36C - Sorting

Prof. Joël Porquet-Lupine

UC Davis - Spring Quarter 2020



Introduction

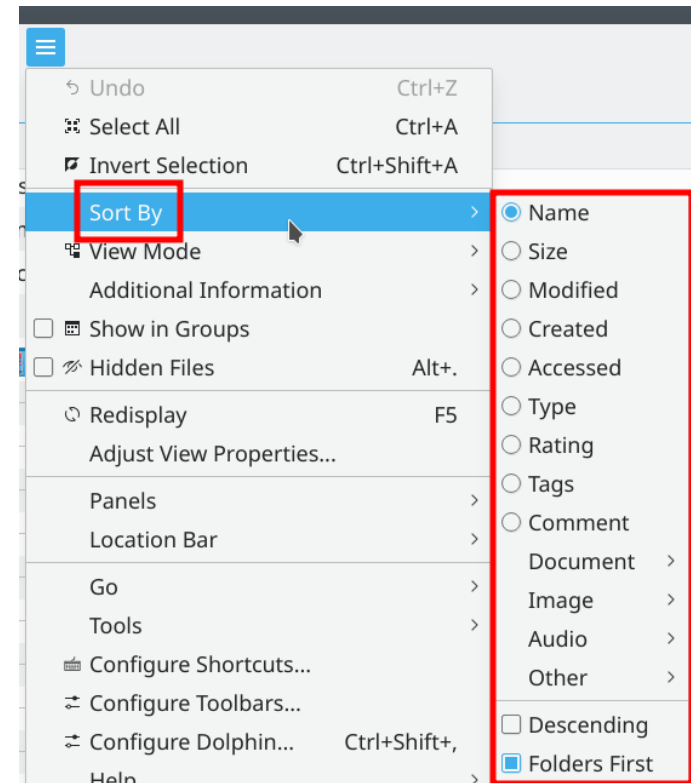
Sorting in the wild

- Sorting is a fundamental operation
 - User interface
 - Data organization
 - Also at the base of other operations...

Sorting-based operations

Once a collection of items is sorted:

- **Searching**
 - Use binary search
- **Element uniqueness**
 - Scan items for adjacent duplicates
- **Frequency distribution**
 - Scan items and count duplicates
- **Selection**
 - Find k th largest item at k th position
- Etc.



Introduction

Definition

- Puts the elements of a list in a certain order
 - E.g., numerical order, lexicographical order
- Elements are *comparable keys*

History

- Sorting algorithms have been a popular research topic since the beginning of computing, and for many decades
 - 50s: e.g., Merge sort, Bubble sort
 - 60s: e.g., Heapsort
 - 70s: e.g., Quicksort
 - Now somewhat considered a solved problem

- Overabundance of possibilities...

Bubble sort, Cocktail shaker sort, Odd-even sort, Comb sort, Gnome sort, Quicksort, Slowsort, Stooge sort, Bogosort, Selection sort, Heapsort, Smoothsort, Cartesian tree sort, Tournament sort, Cycle sort, Weak-heap sort, Insertion sort, Shellsort, Splaysort, Tree sort, Library sort, Patience sorting, Merge sort, Cascade merge sort, Oscillating merge sort, Polyphase merge sort, American flag sort, Bead sort, Bucket sort, Burstsor, Counting sort, Pigeonhole sort, Proxmap sort, Radix sort, Flashsort, etc.

Introduction

Classifications (1/2)

Complexity

Three main classes of complexities:

- Naive algorithms (e.g., Selection sort, Insertion sort)
 - Average complexity of $O(n^2)$
 - Relevant only for small arrays
- Efficient algorithms (e.g., Heapsort, Merge sort, Quicksort)
 - Average complexity of $O(n \log n)$
 - Default option in most cases
- Non-comparison based algorithms (e.g., Bucket sort, Radix sort)
 - Leverage properties of the keys to be sorted
 - Not limited by $O(n \log n)$ bound

Memory occupation

- *In-place* sorting
 - Transform initial collection directly, without using extra space
- Otherwise, extra space often either $O(\log n)$ to $O(n)$

Introduction

Classifications (2/2)

Stability

- A sorting algorithm is stable if equal keys remain in the same relative order before and after the sort
- Important if items have other comparable attributes
 - Equality might not then mean exact duplicates
 - Enables chaining several levels of sorting on the same collection (using a first attribute, then another, etc.)

Example

```
{ { 7, "heart" },  
  { 9, "clubs" },  
  { 3, "diamonds" },  
  { 7, "spades" } }
```

Initial collection

```
{ { 3, "diamonds" },  
  { 7, "heart" },  
  { 7, "spades" },  
  { 9, "clubs" } }
```

Result of stable sorting

```
{ { 3, "diamonds" },  
  { 7, "spades" },  
  { 7, "heart" },  
  { 9, "clubs" } }
```

Potential result of unstable sorting

Selection sort

Principle

- Assuming a collection a of n items, requires n phases (from 0 to $n - 1$)
- At phase i , find index $\min$ of minimum item in the remaining items
- Swap $a[i]$ and $a[\min]$

Snapshot example

i	$\min$	$a[0]$	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	$a[6]$
3	5	A	C	D	U	V	I	S

- Before phase 3:
 - Range 0 to $i - 1$ ($a[0-2]$) is already in sorted and final order
- At phase 3:
 - Find minimum item in range i to $n - 1$ ($a[3-6]$)
 - Swap minimum item ($a[5] = 'I'$) with item at position i ($a[3]$)
- After phase 3:
 - Range 0 to i ($a[0-3]$) is now sorted and in final order
 - Go to phase 4...

Selection sort

Full example

- In **red**, minimum item at phase i
- In **gray**, items in sorted and final order

i	min	a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]
		U	C	D	A	V	I	S
0	3	U	C	D	A	V	I	S
1	1	A	C	D	U	V	I	S
2	2	A	C	D	U	V	I	S
3	5	A	C	D	U	V	I	S
4	6	A	C	D	I	V	U	S
5	5	A	C	D	I	S	U	V
6	6	A	C	D	I	S	U	V
		A	C	D	I	S	U	V

Selection sort

Code

```
// Generic selection sort function
template<typename K>
void selection_sort(std::vector<K> &a) {
    // N phases, from 0 to N-1
    for (unsigned i = 0; i < a.size(); i++) {
        // Assume the minimum is the first element of the remaining collection
        unsigned min = i;

        // Look for a better minimum in remaining collection if any
        for (unsigned j = i + 1; j < a.size(); j++)
            if (a[j] < a[min])
                min = j;

        // Swap minimum with current element
        if (min != i)
            std::swap(a[min], a[i]);
    }
}
```

selection_sort.cc

```
// Vector of unordered integer numbers
std::vector<int> vecint = {
    74, 98, 83, 52, 66, 64, 75, 8, 1, 69
};
...
// Sort vector
selection_sort<int>(vecint);
```

selection_sort.cc

Execution

```
Before : 74 98 83 52 66 64 75 8 1 69
After  : 1 8 52 64 66 69 74 75 83 98
```


Selection sort

Conclusion

- One of the simplest sorting algorithms
- Constant $\Theta(n^2)$ time complexity
 - Even if collection is already sorted!
- In-place
- *Not* stable
 - Long distance exchanges might invert position of equal items

i	min	a[0]	a[1]	a[2]
0	2	B1	B2	A
1	1	A	B2	B1
2	2	A	B2	B1
		A	B2	B1

Insertion sort

Principle

- Also n phases (from 0 to $n - 1$)
- At phase i , swap item at $a[i]$ with each larger on its left

Snapshot example

i	pos	a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]
2'	-	C	D	U	A	V	I	S
3	0	A	C	D	U	V	I	S

- Before phase 3 (i.e., after phase 2, as shown above):
 - Range 0 to $i - 1$ ($a[0-2]$) is locally sorted
 - Range i to $n - 1$ ($a[3-6]$) has not been seen yet
- At phase 3:
 - Make item at position i ($a[3] = 'A'$) find its position in range 0 to $i - 1$ by successive swapping
- After phase 3:
 - Range 0 to i ($a[0-3]$) is now locally sorted
 - Go to phase 4...

Insertion sort

Full example

- In gray, items do not move
- In red, item at phase i finding its proper position in the left range

i	pos	a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]
		U	C	D	A	V	I	S
0	0	U	C	D	A	V	I	S
1	0	C	U	D	A	V	I	S
2	1	C	D	U	A	V	I	S
3	0	A	C	D	U	V	I	S
4	4	A	C	D	U	V	I	S
5	3	A	C	D	I	U	V	S
6	4	A	C	D	I	S	U	V
		A	C	D	I	S	U	V

Insertion sort

Code

```
// Generic insertion sort function
template<typename K>
void insertion_sort(std::vector<K> &a) {
    // N phases, from 0 to N-1
    for (unsigned i = 0; i < a.size(); i++) {
        // Swap current item with already sorted items
        // until finding its correct position
        for (int j = i; j > 0; j--)
            if (a[j] < a[j-1])
                std::swap(a[j], a[j-1]);
            else
                break;
    }
}
```

insertion_sort.cc

Execution

```
// Vector of unordered integer numbers
std::vector<int> vecint = {
    74, 98, 83, 52, 66, 64, 75, 8, 1, 69
};
...
// Sort vector
insertion_sort<int>(vecint);
```

insertion_sort.cc

Before : 74 98 83 52 66 64 75 8 1 69
After : 1 8 52 64 66 69 74 75 83 98

Insertion sort

Conclusion

- With selection sort, one of the simplest sorting algorithms
 - Popular on small arrays, performs better than efficient sorting algorithms
- Usually more efficient than selection sort
 - Best case of $O(n)$ if collection is already sorted
 - However, number of writes is $O(n^2)$ because of the pairwise swaps (versus $O(n)$ for selection sort)
- In-place and stable
- *Online*
 - Can sort collection as it receives it!

Heapsort

Principle

- Two step algorithm:
 1. Build heap out of collection of items ($O(n)$ using Floyd's linear approach)
 2. Remove items in order (n operations, $O(\log n)$ each)
- Total time complexity of $O(n \log n)$

Naive implementation based on min-heap *

```
// Generic heapsort function based on min-heap
template <typename K>
void heapsort(std::vector<K> &a) {
    // Make a vector copy for the heap
    std::vector<K> a_heap(a);
    // Heapify
    BuildHeap(a_heap);
    // Extract min values in order
    for (unsigned i = 0; i < a.size(); i++) {
        a[i] = Top(a_heap);
        Pop(a_heap);
    }
}
```

heapsort_minheap.cc

- Extra vector copy required ($O(n)$)
 - What very specific behavior of `Pop()` can we leverage to avoid it?

* FYI, about 50 lines of code to implement heap methods (based on binary heap from priority queue)

Heapsort

In-place implementation based on max-heap

```
// Generic heapsort function based on max-heap
template <typename K>
void heapsort(std::vector<K> &a) {
    // Heapify vector
    std::make_heap(a.begin(), a.end());
    // Pop max values in order
    for (unsigned i = 0; i < a.size(); i++)
        std::pop_heap(a.begin(), a.end() - i);
}
```

heapsort_maxheap.cc

- At each iteration, max value is pushed at the end of the remaining heap
- At the end, max values have been pushed in descending order from the end of the vector

Conclusion

- Popular sorting algorithm
 - E.g., default sort function in the Linux kernel
- Very consistent sorting algorithm
 - $O(n \log n)$ in average and worst cases
- In-place (if cleverly implemented)
- *Not stable*

ECS 36C - Sorting (part 2)

Prof. Joël Porquet-Lupine

UC Davis - Spring Quarter 2020



Recap

Sorting

- Running time complexity
 - Naive vs efficient vs non-comparison based algorithms
- Memory occupation
 - In-place vs extra space
- Stability
 - Keep equal keys in same initial relative order

Selection sort

- Find minimum in remaining collection and swap with item at index i
- $\Theta(n^2)$, in-place, not stable

Insertion sort

- Swap item at index i with each larger item on its left
- $O(n^2)$, in-place, stable, online

Heapsort

- Build heap out of items, pop all items in order
- $O(n \log n)$, in-place, not stable

Towards faster sorting algorithms

Simple sorting

Name	Best	Average	Worst	Memory	Stable
Selection sort	n^2	n^2	n^2	1	No
Insertion sort	n	n^2	n^2	1	Yes

- Need another paradigm than repetitively and linearly scanning items...

Divide-and-conquer algorithms

1. Divide the work into smaller chunk recursively
 - Make smaller pieces of the larger problem
2. Conquer the individual pieces
3. Combine the results back up recursively

```
DivideAndConquer (input) {  
    if (input small enough to solve)  
        conquer, solve, return results  
    else  
        divide input into smaller pieces  
        DivideAndConquer(smaller piece #1)  
        DivideAndConquer(smaller piece #2)  
        combine both results and return  
}
```

- Divide-and-conquer based sorting algorithms:
 - Merge sort
 - Quicksort

Merge sort

Principle

Divide

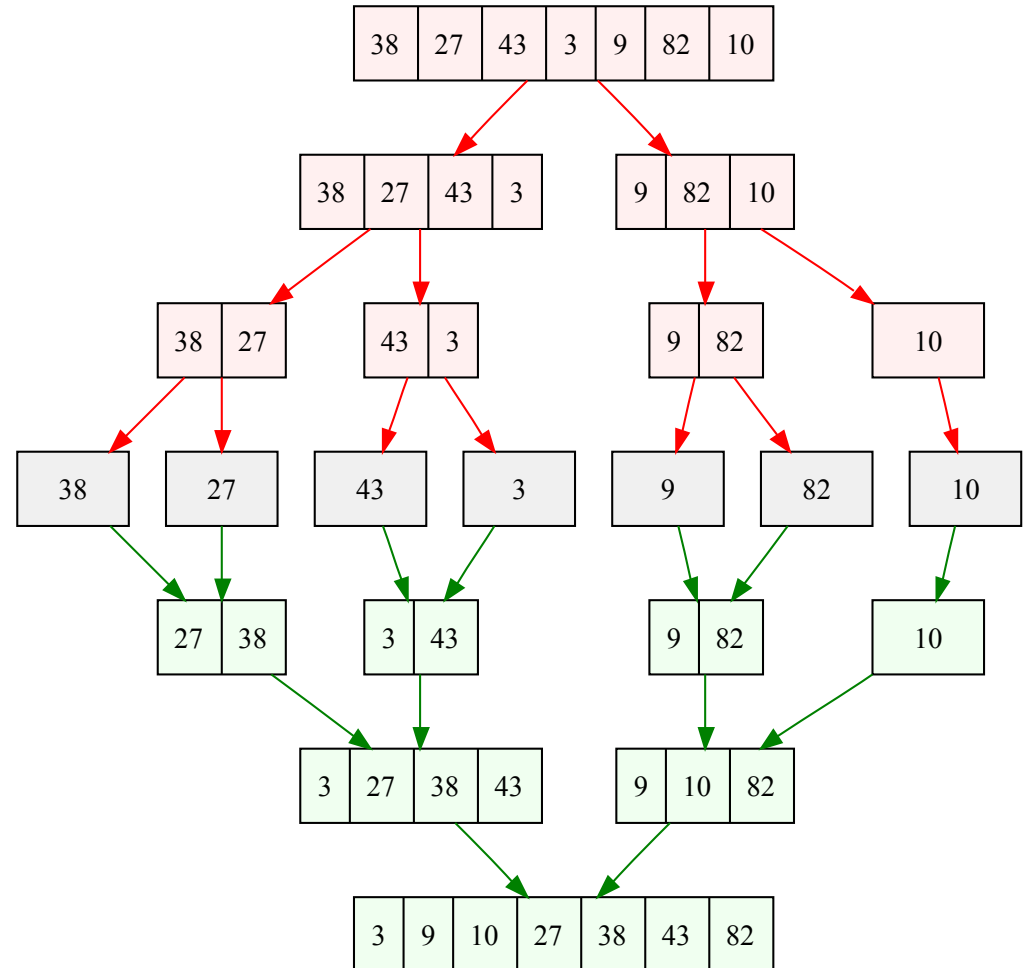
- If a sub-array has more than 1 item, recursively divide it into two halves

Conquer

- If a sub-array has 0 or 1 item, it is sorted

Combine

- Merge two adjacent sorted sub-arrays together
- Recursively merge until whole array is reconstituted

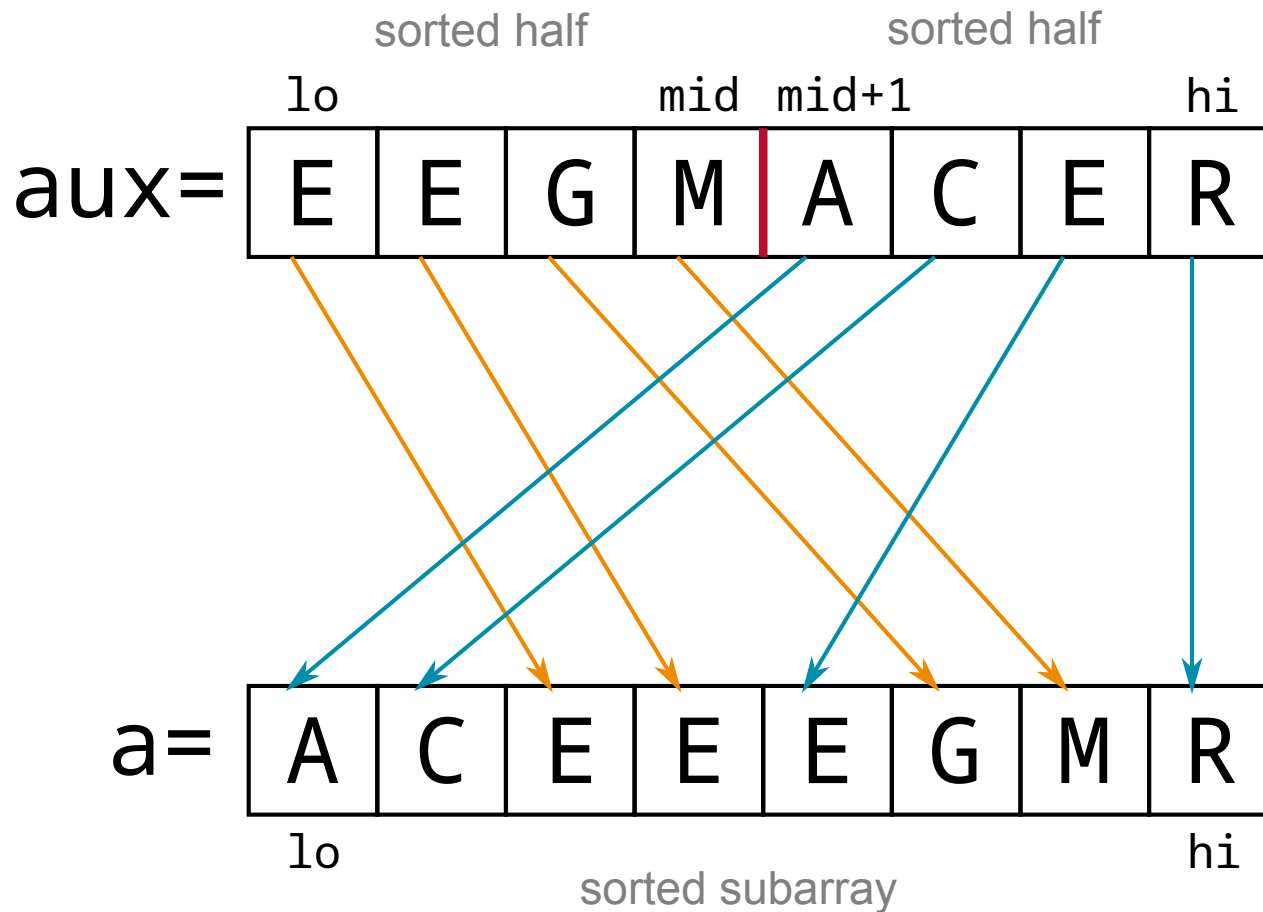


Merge sort

Merging sorted halves

Example

- Merge two sorted halves $\text{aux}[\text{lo}]$ to $\text{aux}[\text{mid}]$, and $\text{aux}[\text{mid}+1]$ to $\text{aux}[\text{hi}]$, into a single sub-array $\text{a}[\text{lo}]$ to $\text{a}[\text{hi}]$



Merge sort

Merging code

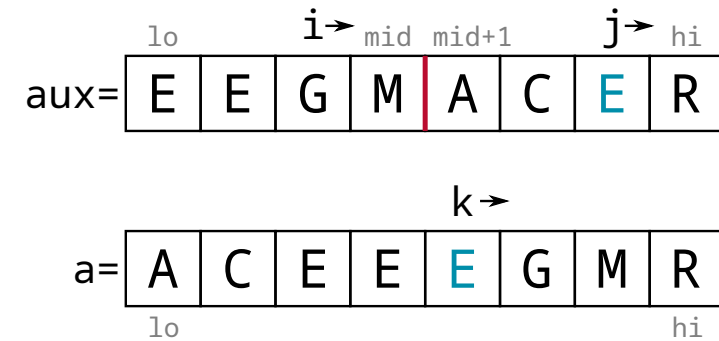
```
template<typename K>
void merge_sort_merge(std::vector<K> &a,
                     std::vector<K> &aux,
                     int lo, int mid, int hi) {
    // Save both halves into auxiliary space
    for (int k = lo; k <= hi; k++)
        aux[k] = a[k];

    // Merge the two halves: [lo,mid][mid+1,hi]
    int i = lo, j = mid + 1;
    for (int k = lo; k <= hi; k++) {
        // Left half has been entirely merged
        if (i > mid)
            // Just take what's remaining from right half
            a[k] = aux[j++];

        // Right half has been entirely merged
        else if (j > hi)
            // Just take what's remaining from left half
            a[k] = aux[i++];

        // Get smallest item from either left or right halves
        else if (aux[j] < aux[i])
            a[k] = aux[j++];
        else
            a[k] = aux[i++];
    }
}
```

merge_sort.cc



Merge sort

Dividing into halves

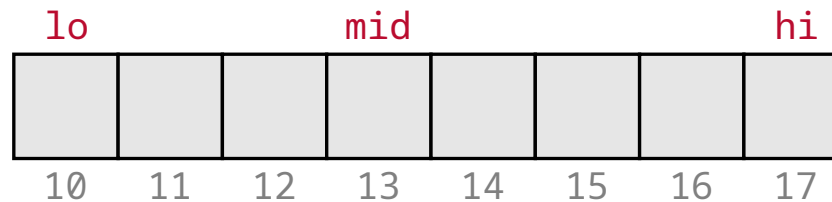
```
template<typename K>
void merge_sort_recur(std::vector<K> &a,
                     std::vector<K> &aux,
                     int lo, int hi) {
    // Recursion stop condition: subarray is sorted if it contains 0 or 1 item
    if (hi <= lo)
        return;

    // Otherwise, further divide subarray into two halves
    int mid = lo + (hi - lo) / 2;

    // Recursively sort right half
    merge_sort_recur(a, aux, lo, mid);
    // Recursively sort left half
    merge_sort_recur(a, aux, mid + 1, hi);

    // Merge two halves
    merge_sort_merge(a, aux, lo, mid, hi);
}
```

merge_sort.cc



Merge sort

Putting the pieces together

```
template<typename K>
void merge_sort_merge(std::vector<K> &a,
                     std::vector<K> &aux,
                     int lo, int mid, int hi) {
    ...
}

template<typename K>
void merge_sort_recur(std::vector<K> &a,
                     std::vector<K> &aux,
                     int lo, int hi) {
    ...
}

// Generic merge sort function
template<typename K>
void merge_sort(std::vector<K> &a) {
    // Need auxiliary space of the same size
    std::vector<K> aux(a.size());
    merge_sort_recur(a, aux, 0, a.size() - 1);
}
```

merge_sort.cc

Execution

```
std::vector<int> vecint = {
    74, 98, 83, 52, 66,
    64, 75, 8, 1, 69
};
...
// Sort vector
merge_sort<int>(vecint);
```

merge_sort.cc

```
Before : 74 98 83 52 66 64 75 8 1 69
After  : 1 8 52 64 66 69 74 75 83 98
```

Merge sort

Conclusion

- Popular sorting algorithm
 - Compete directly with quicksort
 - Best choice in certain situations: e.g., sorting a linked list
- Consistent sorting algorithm
 - $O(n \log n)$ in average and worst cases
 - Dividing phase ($O(n)$), conquering/merging phase ($O(n \log n)$)
- Not in-place
 - Memory space occupation of $O(n)$
- Stable

Quicksort

Principle

Divide

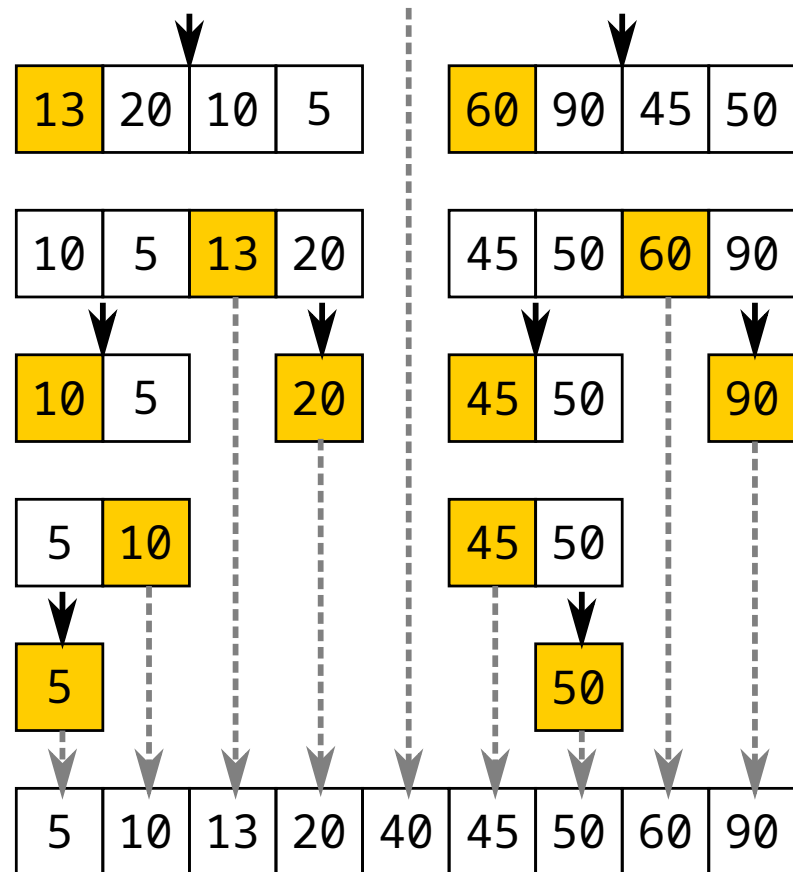
- If partition has 0 or 1 item, it is sorted
- If partition has more than 1 item
 - Pick a pivot (e.g., first value)
 - Rearrange partition around pivot

Conquer

- Pivot is in final position
- Recursively sort right and left sub-partitions around pivot

40	90	60	5	13	10	20	45	50
----	----	----	---	----	----	----	----	----

13	20	10	5	40	60	90	45	50
----	----	----	---	----	----	----	----	----

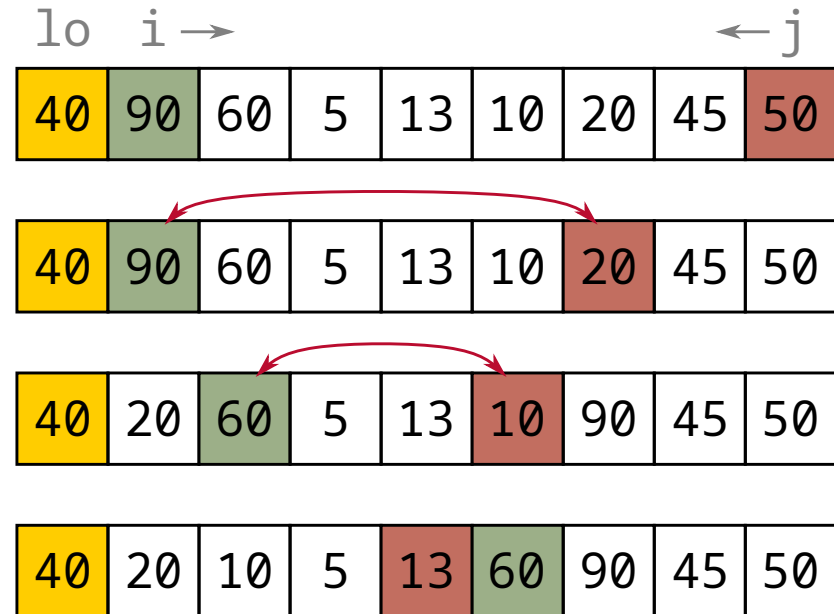


Quicksort

Partitioning

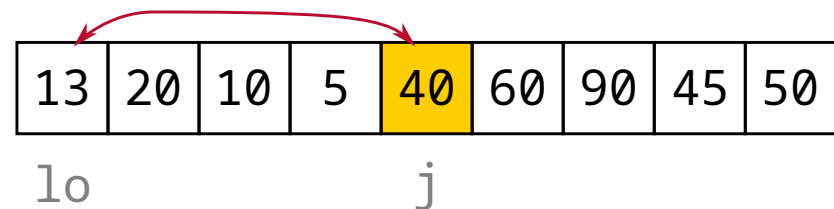
Phase 1: partition according to pivot

- Pivot is $a[lo]$
- Repeat until i and j pointers cross
 - Scan i from left to right while $a[i] < a[lo]$
 - Scan j from right to left while $a[j] > a[lo]$
 - Exchange $a[i]$ with $a[j]$



Phase 2: place pivot in right position

- When pointers cross
 - Exchange $a[lo]$ with $a[j]$
- Pivot is now in final position in sorted array



Quicksort

Partitioning code

```
template<typename K>
int quicksort_partition(std::vector<K> &a,
                       int lo, int hi) {
    int i = lo, j = hi + 1;
    while (true) {
        // From left to right, find first item
        // larger than pivot
        while (a[++i] < a[lo])
            // Prevents from going out of bounds
            if (i == hi) break;

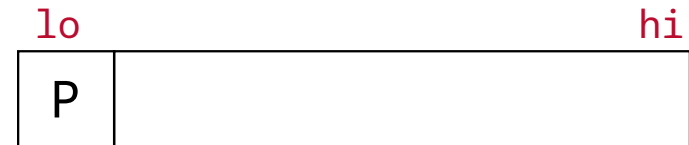
        // From right to left, find first item
        // smaller than pivot
        while (a[lo] < a[--j])
            // To keep code symmetry but unnecessary
            if (j == lo) break;

        // If pointers i and j have crossed, stop
        if (i >= j) break;
        // Otherwise swap items
        std::swap(a[i], a[j]);
    }

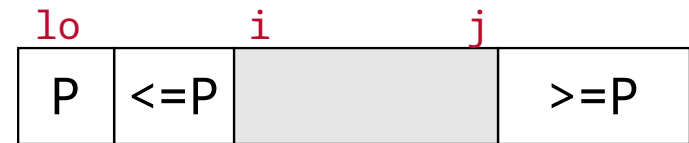
    // Put pivot in final position
    std::swap(a[lo], a[j]);
    // Return pivot position
    return j;
}
```

quicksort.cc

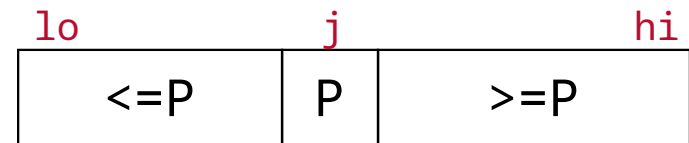
Before



During



After



Quicksort

Recursive sorting

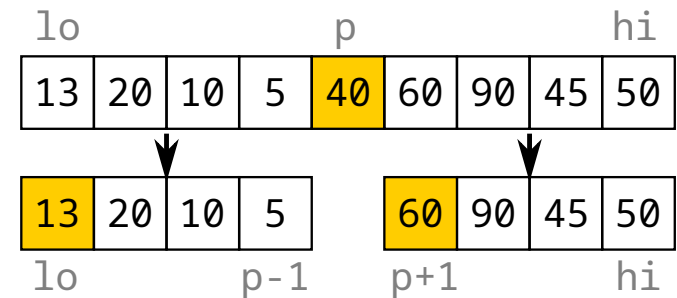
Code

```
template<typename K>
void quicksort_recur(std::vector<K> &a,
                    int lo, int hi) {
    // Recursion stop condition:
    // Sub-array is sorted if it contains 0 or 1 item
    if (hi <= lo)
        return;

    // Partition array and get resulting pivot pos
    int p = quicksort_partition(a, lo, hi);
    // Recursively sort left sub-partition
    quicksort_recur(a, lo, p - 1);
    // Recursively sort right sub-partition
    quicksort_recur(a, p + 1, hi);
}

// Generic quicksort function
template<typename K>
void quicksort(std::vector<K> &a) {
    quicksort_recur(a, 0, a.size() - 1);
}
```

quicksort.cc



Execution

```
std::vector<int> vecint = {
    74, 98, 83, 52, 66,
    64, 75, 8, 1, 69
};
...
// Sort vector
quicksort<int>(vecint);
```

quicksort.cc

Before : 74 98 83 52 66 64 75 8 1 69
After : 1 8 52 64 66 69 74 75 83 98

Quicksort

Conclusion

- One of the most popular sorting algorithms
- Efficient but not guaranteed
 - On average $O(n \log n)$
 - But worst case is $O(n^2)$ (seems rare)
 - Choice of pivot is important in the performance
 - Choosing leftmost item as pivot can cause worst-case behavior for already sorted arrays
 - Possible strategy is to shuffle the array before sorting it
- Memory occupation
 - $O(\log n)$ for the stack space
- In-place but not stable
 - Possibility to make it stable if using auxiliary array ($O(n)$ extra space)

Bucket sort

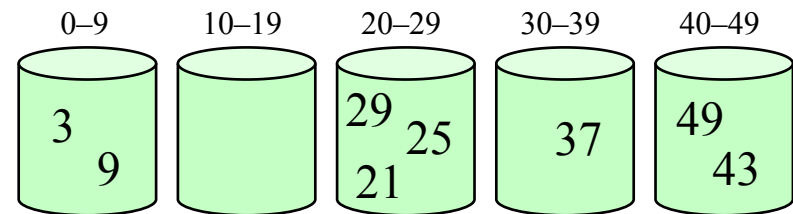
Principle

- Distribute all the items of an array into buckets
- Sort the buckets individually
- Recombine buckets in order into original array

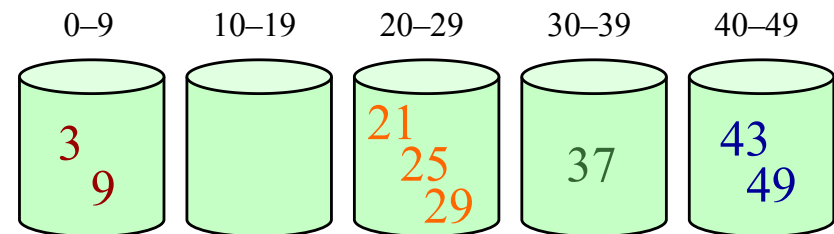
Example

- Array of numbers to be sorted
- Scatter into buckets

29, 25, 3, 49, 9, 37, 21, 43



- Sort each bucket individually



- Gather sorted buckets in order

3 9 21 25 29 37 43 49

Bucket sort

Code

```
// Bucket sort for int between 00 and 99
void bucket_sort(std::vector<int> &a) {
    // One bucket for each tens
    std::vector<std::vector<int>> bckts(10);

    // Scatter items in buckets
    for (auto &i : a) {
        // Extract tens digit from item
        int bi = (i / 10) % 10;
        // Put item into corresponding bucket
        bckts[bi].push_back(i);
    }

    // Sort each bucket separately
    for (auto &b : bckts)
        insertion_sort(b);

    // Combine all buckets back together
    int k = 0;
    for (auto &b : bckts)
        for (auto &i : b)
            a[k++] = i;
}
```

bucket_sort.cc

Bucket sort

Limitations and conclusion

- Can work well if
 - Items are positive integers
 - Belong to a well-defined (usually small) range
 - And are uniformly distributed in the range
- Best-case scenario
 - If number of buckets is equal to the number of items
 - And items are uniformly distributed across the buckets
 - Then complexity of $O(n)$
- Worst-case scenario
 - All the items end up in a single bucket
 - Then complexity of bucket sorting
 - Up to $O(n^2)$ if using simple sorting algorithm

Radix sort

Principle (*least significant digit radix sort*)

- Starting with least significant digit, put each item in corresponding bucket
- Repeat on next digit, until reaching most significant digit

Example

- Unordered list of numbers between 0-999: 170, 45, 75, 90, 12, 802, 2, 66

- (1) Put each item in bucket corresponding to their unit digit

0s	1s	2s	3s	4s	5s	6s	7s	8s	9s
170, 90		12, 802, 2			45, 75	66			

170, 90, 12, 802, 2, 45, 75, 66

- (2) Repeat on tens digit

0s	1s	2s	3s	4s	5s	6s	7s	8s	9s
802, 02	12			45		66	170, 75		90

802, 2, 12, 45, 66, 170, 75, 90

- (3) Repeat on hundreds digit

0s	1s	2s	3s	4s	5s	6s	7s	8s	9s
002, 012, 045, 066, 075, 090	170							802	

Conclusion

Hybrid sorting

For example, *introsort*:

- Starts with quicksort
- Resorts to heapsort when recursion becomes too deep
- Uses simple insertion sort for small partitions

New sorting algorithms

- Timsort (2002)
 - Used by Python and Java
 - Leverages the fact that most collections usually have consecutive ordered items already
- Library sort (2006)
 - Variant of insertion sort that leaves gaps to accelerate subsequent insertions of new items

Standard language libraries

- C++: `std::sort()`
- C: `qsort()`
- Python: `list.sort()`
- Java: `java.util.Collections.sort()`